



ESCUELA PROFESIONAL DE INGENIERÍA DE COMPUTACIÓN Y SISTEMAS

INFORME DE CONFIGURACIÓN DE VERCEL

**PROYECTO:
AGROSMART INSIGHTS – OPTIMIZACIÓN DE MERCADOS Y PRECIOS
AGRÍCOLAS**

**CURSO:
TALLER DE PROYECTOS**

**PROFESOR:
ING. NORMA VIRGINIA LEÓN LESCANO**

**INTEGRANTE:
León Cangalaya, Gabriel Emilio
Líder DevSecOps – Célula 2**

FECHA: 25/08/2026

LIMA – PERÚ

INFORME DE CONFIGURACIÓN DE VERCEL – AGROSMART INSIGHTS

1. Objetivo

El presente informe tiene por objetivo documentar, sustentar y verificar la configuración del despliegue del frontend de AgroSmart Insights en la plataforma Vercel durante el Sprint 1. El alcance técnico abarca la importación del repositorio oficial desde GitHub, la delimitación del subdirectorío raíz (Root Directory) en una arquitectura monorepo, la validación del archivo de configuración vercel.json, la parametrización de variables de entorno para el enlace con el backend n8n en Render y la comprobación de la integración continua.

1.1. Fundamento metodológico y enfoque DevSecOps

La arquitectura de AgroSmart Insights implementa un desacoplamiento estricto entre la interfaz de usuario desarrollada en React con Vite y los servicios de orquestación analítica e inteligencia artificial alojados en Render. Este modelo arquitectónico demanda que la capa frontend sea servida a través de una red perimetral de distribución global (Edge Network), lo cual optimiza los tiempos de carga, minimiza la latencia de respuesta y reduce la superficie de ataque al no exponer servidores de aplicación directos (Vercel, 2026).

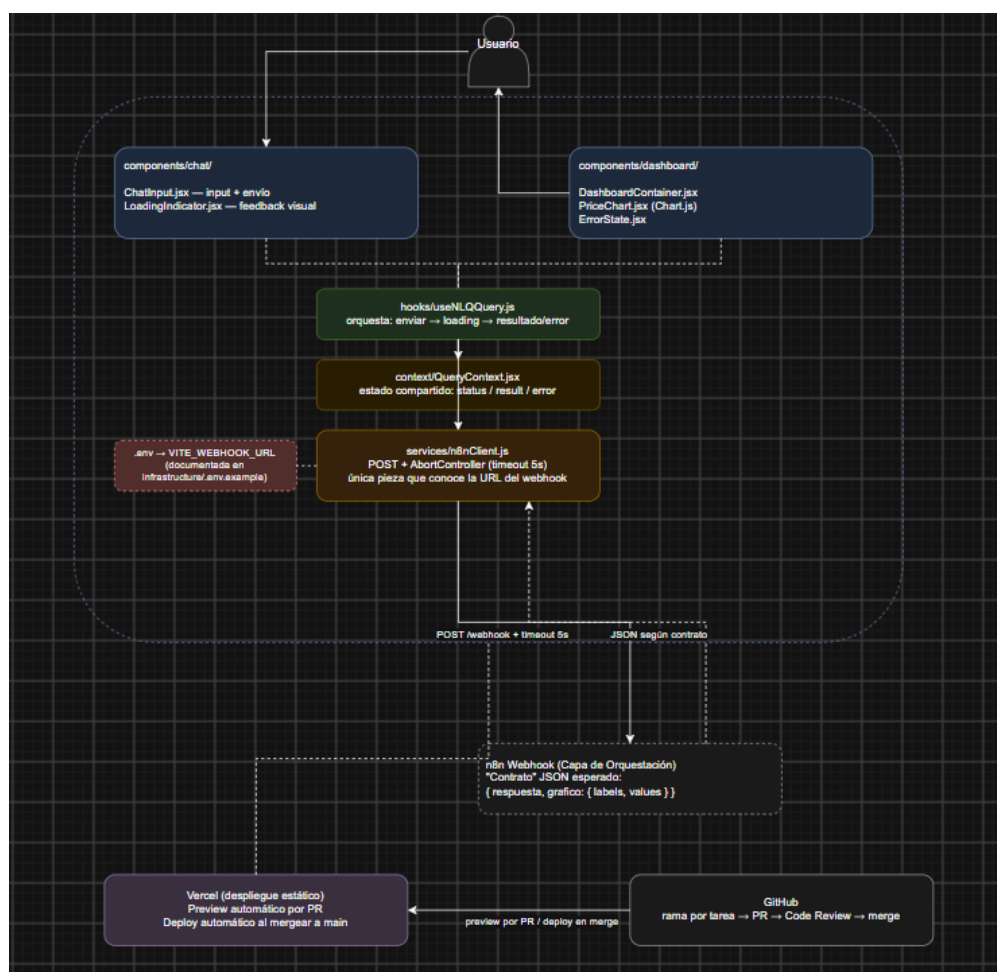
Desde la perspectiva de gobernanza y seguridad DevSecOps, se aplican los principios de entrega segura del marco NIST SP 800-218. Cada modificación introducida en la interfaz gráfica debe ser evaluada automáticamente mediante el pipeline validate-frontend en GitHub Actions antes de ser admitida en la rama main, impidiendo la introducción de vulnerabilidades de código, malas prácticas de sintaxis o filtración accidental de credenciales (NIST, 2022).

1.2. Arquitectura de despliegue frontend

El módulo frontend agrosmart-frontend ofrece una interfaz de lenguaje natural (NLQ) orientada a productores agrícolas y comerciantes del Gran Mercado Mayorista de Lima (GMMML). Las peticiones emitidas por los usuarios son procesadas por el cliente asíncrono `n8nClient.js` y transmitidas mediante peticiones HTTP POST seguras hacia el endpoint webhook del workflow WF2 desplegado en Render.

Figura 1

Arquitectura de comunicación desacoplada entre Frontend Vercel y Backend Render



Nota. Diagrama que ilustra la separación funcional de capas. Los clientes acceden a la aplicación React servida por la red Edge de Vercel, la cual se comunica asincrónicamente con el webhook expuesto por

n8n sobre PostgreSQL en Render. *Fuente:* Elaboración propia a partir del diseño de arquitectura de AgroSmart Insights (Equipo 2 AgroSmart Insights, 2026).

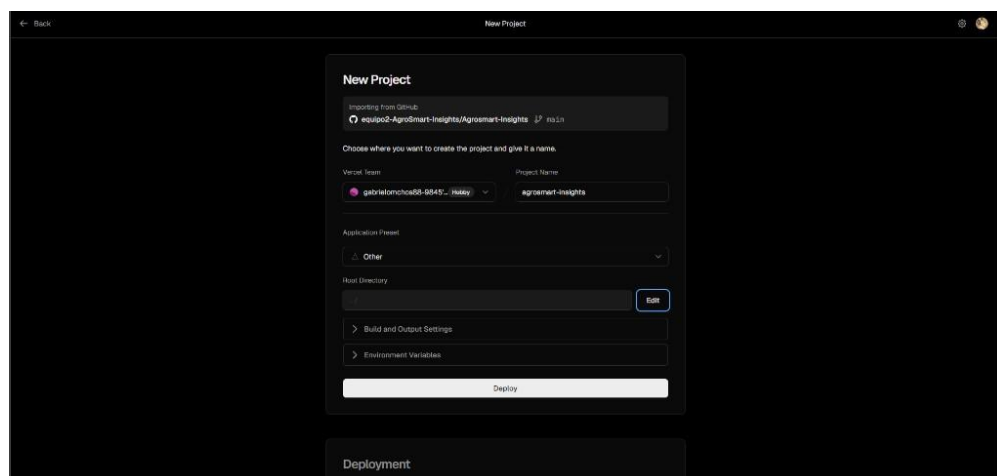
2. Procedimiento de configuración en Vercel

2.1. Conexión e importación del repositorio

Se inició sesión en la consola de administración de Vercel (<https://vercel.com>) vinculada con la cuenta de la organización institucional en GitHub. Se seleccionó la opción Add New → Project y se localizó el repositorio equipo2-AgroSmart-Insights/Agrosmart-Insights, estableciendo main como la rama de producción predeterminada.

Figura 2

Importación del repositorio Agrosmart-Insights en la plataforma Vercel



Nota. Captura de pantalla de la consola de Vercel en la sección Add New Project. Se visualiza la selección del repositorio institucional, la rama de producción main y el nombre del proyecto agrosmart-insights. *Fuente:* Elaboración propia a partir de la consola de Vercel (Vercel, 2026).

2.2. Configuración del Root Directory en entorno Monorepo

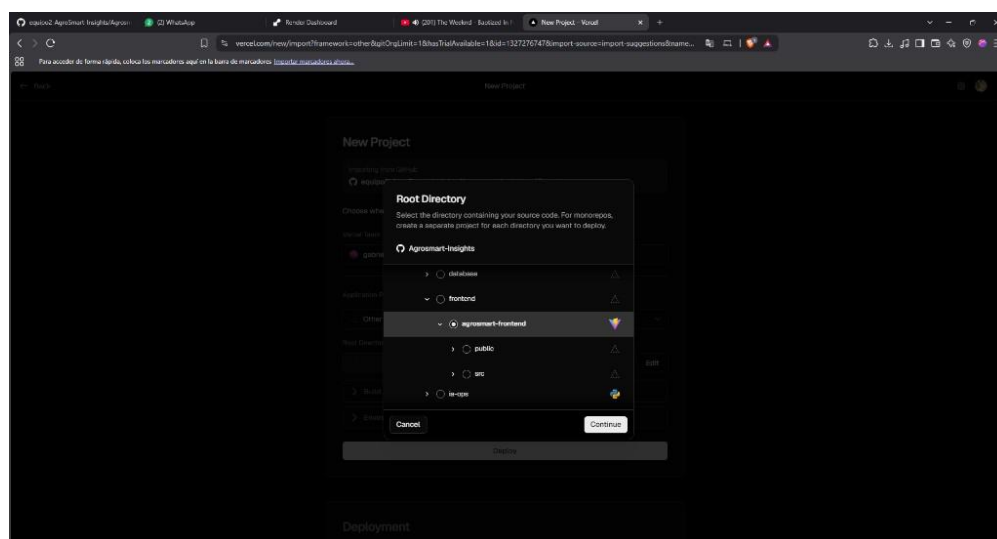
El repositorio de AgroSmart Insights está estructurado como un monorepo que alberga simultáneamente el código del frontend, flujos de trabajo de n8n, migraciones

de base de datos y scripts de automatización. Debido a que la aplicación React no se localiza en la raíz del repositorio (./), fue necesario ajustar el parámetro Root Directory.

Utilizando el selector interactivo de directorios de Vercel, se navegó hacia el subdirectorio `src/frontend/agrosmart-frontend`. Al confirmar dicha ubicación, el motor de análisis de Vercel detectó automáticamente el archivo `package.json` y configuró el entorno de compilación optimizado para Vite.

Figura 3

Selección del subdirectorio `src/frontend/agrosmart-frontend` como Root Directory



Nota. Captura de pantalla del selector de directorio raíz de Vercel. Se observa la navegación hacia `src/frontend/agrosmart-frontend` y el reconocimiento automático del framework Vite mediante su ícono distintivo. *Fuente:* Elaboración propia a partir de la consola de Vercel (Vercel, 2026).

2.3. Parámetros de compilación y empaquetado (Build Settings)

Se validó la coherencia de los comandos de construcción con el archivo `vercel.json` presente en el frontend. Los parámetros configurados son los siguientes:

- Framework Preset: Vite (reconocimiento automático).

- Build Command: `npm run build` (genera los artefactos estáticos minificados en el directorio `dist/`).
- Output Directory: `dist` (directorio público distribuido por la red de Vercel).
- Install Command: `npm ci` (garantiza una instalación determinista basada en `package-lock.json`).

2.4. Configuración de Variables de Entorno

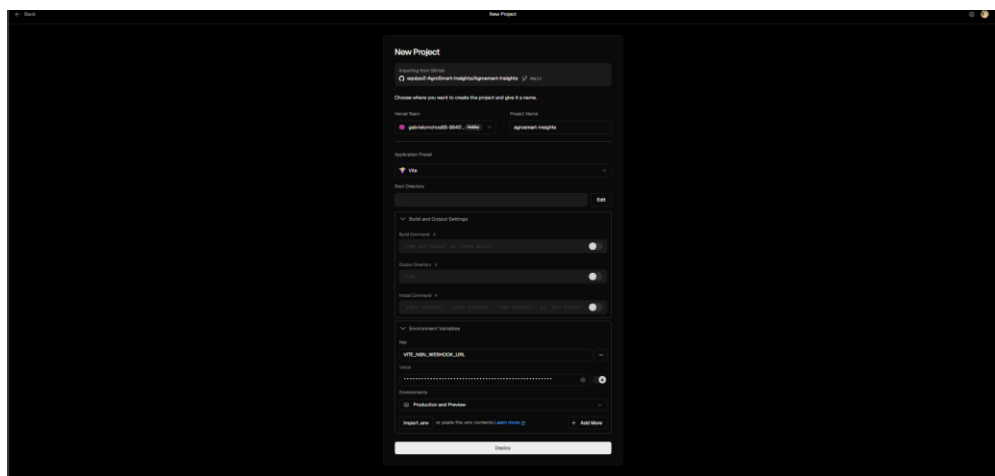
Para habilitar la conectividad en tiempo de ejecución entre la interfaz gráfica y el motor analítico de `n8n`, se registró la variable de entorno en la pantalla `New Project` y, de forma persistente, en `Project Settings` → `Environment Variables`:

- `VITE_N8N_WEBHOOK_URL`: URL pública absoluta del webhook de producción en Render (<https://agrosmart-n8n.onrender.com/webhook/v1/query>).
- Ámbitos: `Production` y `Preview`, de modo que las vistas previas de rama también apunten al backend cloud.

El prefijo `VITE_` permite que el empaquetador inyecte el valor durante la fase de compilación sin comprometer secretos sensibles del servidor. Las llaves de Groq, Gemini, Hugging Face y MapTiler permanecen exclusivamente en Render.

Figura 4

Variable de entorno `VITE_N8N_WEBHOOK_URL` en el despliegue de Vercel



Nota. Captura de la pantalla New Project de Vercel. Se observa el framework Vite, la variable VITE_N8N_WEBHOOK_URL enmascarada y los ámbitos Production y Preview. El Root Directory debe quedar en src/frontend/agrosmart-frontend antes de pulsar Deploy. *Fuente:* Elaboración propia a partir de la consola de Vercel (Vercel, 2026).

3. Verificación, Pipeline CI/CD y Despliegue

3.1. Integración con el pipeline GitHub Actions

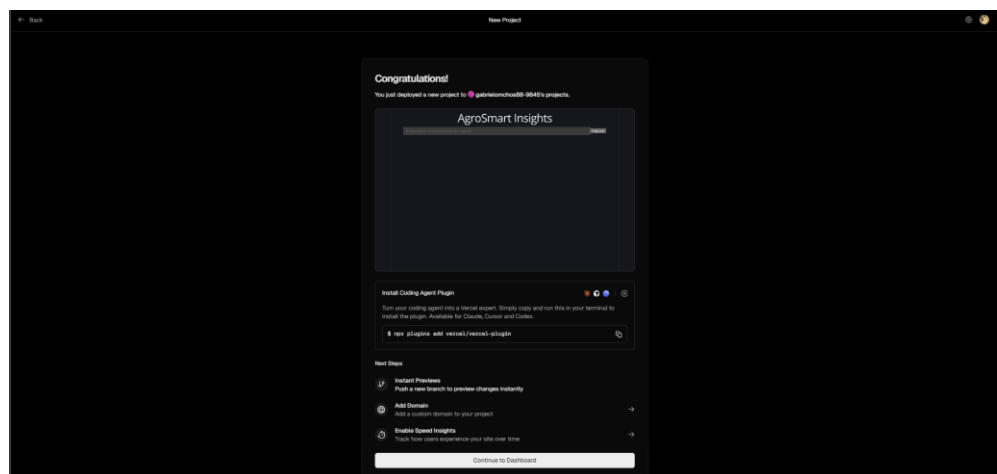
En cumplimiento con las políticas de control de calidad, el flujo de trabajo frontend-ci.yml ejecuta el análisis estático de código (ESLint) y la prueba de empaquetado (npm run build) de forma obligatoria ante cada Pull Request dirigido a main. Asimismo, Vercel genera despliegues de vista previa (Preview Deployments) por cada rama secundaria para facilitar la revisión visual.

3.2. Verificación de despliegue en producción

El 25 de agosto de 2026 se ejecutó el Deploy inicial del proyecto agrosmart-insights. Vercel completó la compilación Vite y mostró la pantalla Congratulations con vista previa de la interfaz AgroSmart Insights, confirmando el estado Ready y el cifrado SSL/TLS automático.

Figura 5

Despliegue exitoso del frontend AgroSmart Insights en Vercel



Nota. Captura de la pantalla Congratulations de Vercel. Se visualiza la vista previa de AgroSmart Insights ya publicada en la red Edge. *Fuente:* Elaboración propia a partir de la consola de Vercel (Vercel, 2026).

3.3. Prueba de extremo a extremo y ajuste de timeout

Con el webhook de n8n ya registrado en producción, se ejecutó una consulta de prueba en el chat: «¿Cuál es el precio actual de la papa?». El frontend abortó a los 6 segundos porque n8nClient.js tenía `TIMEOUT_MS = 6000`. El flujo WF2 en Render (clasificación LLM, consulta PostgreSQL y redacción) tarda habitualmente entre 8 y 20 segundos en el plan gratuito, de modo que el corte no era un fallo de n8n sino un límite del cliente.

Se actualizó el cliente a un timeout de 30 segundos (configurable con `VITE_API_TIMEOUT_MS`) y se documentó el valor en `.env.example`. Este ajuste forma parte del presente Pull Request; hasta que se fusione en main, la prueba E2E completa en Vercel seguirá limitada por los 6 segundos del bundle publicado.

Figura 6

Error de timeout de 6 segundos en el chat publicado en Vercel



Nota. Captura de AgroSmart Insights en producción. La consulta se procesó en n8n, pero el navegador canceló la espera al cumplir el límite de 6 segundos del cliente. *Fuente:* Elaboración propia a partir de la aplicación desplegada en Vercel.

4. Consideraciones de Seguridad DevSecOps

El despliegue en Vercel incorpora las siguientes medidas de protección:

- Aislamiento de claves de IA: Ninguna llave privada (Groq, Gemini, Hugging Face) reside en el bundle del cliente; todas se conservan aisladas en el backend de Render.
- Inmutabilidad de artefactos: Cada despliegue se encuentra asociado al hash de commit criptográfico inalterable de GitHub.
- Tránsito cifrado obligatorio: Se implementa HTTPS estricto con cabeceras HSTS para toda la comunicación web.

5. Conclusión

La puesta en marcha del frontend de AgroSmart Insights en Vercel quedó evidenciada el 25 de agosto de 2026: repositorio importado, Root Directory del monorepo, variable VITE_N8N_WEBHOOK_URL apuntando a Render y build Ready. El único refinamiento pendiente de fusión es el timeout de 30 segundos del cliente NLQ, necesario para que las respuestas de WF2 no se corten en el plan gratuito de Render.

Referencias

Equipo 2 AgroSmart Insights. (2026). Agrosmart-Insights [Repositorio de software].

GitHub. <https://github.com/equipo2-AgroSmart-Insights/Agrosmart-Insights>

GitHub. (s. f.). Acerca de las ramas protegidas. Documentación de GitHub.

<https://docs.github.com/es/repositories/configuring-branches-and-merges-in-your-repository/defining-the-mergeability-of-pull-requests/about-protected-branches>

National Institute of Standards and Technology. (2022). Secure Software Development Framework (SSDF) Version 1.1 (NIST SP 800-218).

<https://csrc.nist.gov/Projects/ssdf>

Red Hat. (s. f.). ¿Qué es DevSecOps? <https://www.redhat.com/es/topics/devops/what-is-devsecops>

Render. (2026). Blueprint specification. <https://render.com/docs/blueprint-spec>

Vercel. (2026). Deploying a Git repository. <https://vercel.com/docs/deployments/git>

Vercel. (2026). Monorepos. <https://vercel.com/docs/monorepos>